Drugi popravni test iz Algoritama i struktura podataka

Prvi praktični test održan 19.02.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test2_GrupaB_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Konverzija matrice u listu susedstva*

Napisati algoritam koji će vršiti direktnu konverziju matrice susedstva u listu susedstva. Algoritam kao ulazni parametar treba da prihvati matricu, a treba da rezultuje kreiranim dinamičkim nizom listi.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***ConvertAdjacencyMatrixToAdjacencyList*** koja treba da sadrži implementaciju konverzije matrice u listu susedstva.
- U *Main* metodi obezbediti način za testiranje i proveru nove funkcionalnosti.

Zadatak 2 – *Karakter sa najdužim nizom bez ponavljanja*

Napisati metodu koja prima string i pronalazi **dužinu najdužeg podstringa** koji se sastoji od **potpuno jedinstvenih karaktera** (bez ijednog ponavljanja).

Dodatni zahtevi za rešavanje zadatka:

- Koristiti Hash tabelu iz šablona za čuvanje broja pojavljivanja svakog karaktera.
- Implementaciju pisati u metodi: ***public static int LengthOfLongestUnique(string str).***
- Vremenska kompleksnost treba da bude **O(n).**
- Ignorisati razliku između malih i velikih slova.

Primer:

Input 1: *str = „abcabcbb"*

Output 1: *3 (To su podstringovi: „abc“, „bca“, „cab“)*

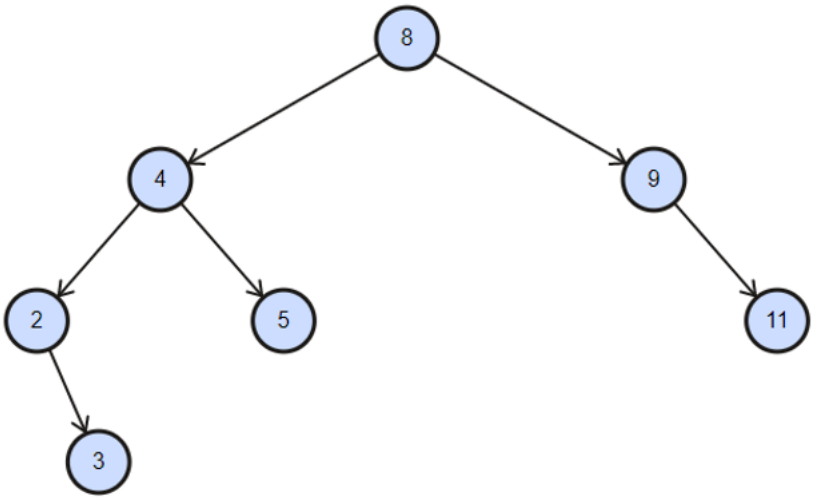
Zadatak 3 – *Provera maksimalne dubine lista (Max Leaf Depth)*

U okviru klase BinarySearchTree napisati metodu koja proverava da li u stablu postoji ijedan list koji se nalazi na dubini većoj od zadate vrednosti k.

Dodatni zahtevi za rešavanje zadatka:

1. Javna metoda: ***public bool hasLeafDeeperThan(int k).***
2. Privatna metoda: ***private bool hasLeafDeeperThan (Node current, int level, int k).***
3. Podrazumeva se da stablo ima barem dva čvora.
4. Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

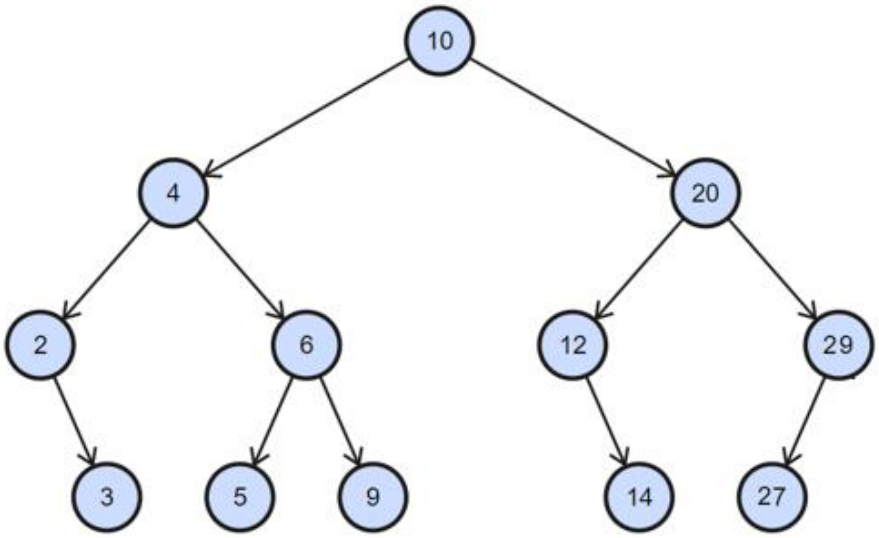
Primeri:



Slika 1 - Primer BST-a

Input 1: *k = 3*

Output 1: *true („3“ se nalazi na toj dubini)*



Slika 2 - Primer BST-a

Input 2: *k = 2*

Output 2: *false (Nijedan list se ne nalazi na toj dubini)*